Administración de Redes
Laboratorio 7 - Dockerfile y docker-compose

Operación Despliegue

Prof. Ricardo Pérez

riperez@utalca.cl

1. Introducción

Una empresa de comercio electrónico acaba de contratar a su equipo como nuevo equipo DevOps. El equipo anterior se fue dejando una tienda en línea construida con **microservicios**: cinco componentes escritos en distintos lenguajes, algunos con Dockerfiles que funcionan mal, otros que no funcionan y otros que directamente no existen. La tienda debe estar **en producción hoy**.

En este laboratorio cada equipo debe empaquetar, desplegar y operar esa tienda, tomando decisiones técnicas reales: qué imagen base usar, cómo cuidar el tamaño y la seguridad de las imágenes, cómo manejar secretos, cómo aislar las redes y cómo reaccionar cuando algo falla. Los equipos compiten entre sí por puntos; los tres mejores obtienen una bonificación de décimas en su nota.

- **Modalidad:** equipos de tres integrantes (si el curso no es múltiplo de tres, algún equipo tendrá cuatro).
- **Duración:** dos sesiones de 2 horas. La primera cubre construcción e integración; la segunda, incidentes, documentación y defensa.
- **Herramientas:** Docker Engine con Docker Compose v2, Git y Python (solo para ejecutar el verificador).

2. Meta de hoy

Al terminar esta sesión, la tienda debe construirse y levantarse completa con `docker compose`. El navegador debe consultar productos, crear un pedido y observar cómo el `worker` lo cambia de `pending` a `processed`. Hoy no se realizan incidentes ni defensa oral, se evaluarán en el horario del laboratorio. Sin embargo, deben quedar evidencias de los commits de cada estudiante.

Criterio de salida: el equipo deja un commit llamado `checkpoint dia 1`, anota sus pendientes y ejecuta el verificador (verify.py). No es obligatorio obtener 50/50 hoy, pero el flujo completo debe quedar funcionando o el bloqueo debe estar diagnosticado.

3. Organización del equipo

Los equipos son formados por los estudiantes al comenzar la clase y deben tener tres integrantes. Se permite un equipo de cuatro únicamente cuando la cantidad total del curso lo haga necesario. Una persona no puede pertenecer a dos equipos.

Trabajen en ramas o hagan commits pequeños para no sobrescribir cambios. Antes de comenzar, cada integrante configura su identidad:

```
git config user.name "Tu Nombre"
git config user.email "tu.correo@alumnos.utalca.cl"
```

Rol inicial	Primera responsabilidad	Después revisa
A	`catalog` y `worker`	Dockerfile y cambios del rol B
B	`orders` y `frontend`	Dockerfile y cambios del rol A
C	`.dockerignore`, seguridad y verificador	`docker-compose.yml` con A y B

Cada integrante debe contribuir en al menos dos áreas. El líder coordina una tarea; no trabaja aislado.

4. Arquitectura mínima que deben comprender

- El navegador accede únicamente a `frontend` por el puerto asignado en `.env`.
- `frontend` reenvía `/api/products` a `catalog` y `/api/orders` a `orders`.
- `catalog` consulta PostgreSQL.
- `orders` guarda el pedido en PostgreSQL y lo encola en Redis.
- `worker` consume la cola de Redis y actualiza el pedido en PostgreSQL.

No cambien los nombres `frontend`, `catalog`, `orders`, `worker`, `db` y `cache`: también son nombres DNS dentro de las redes de Compose.

5. Archivos que pueden modificar

- Dockerfiles y `.dockerignore` de los cuatro servicios de aplicación.
- `docker-compose.yml`, `.env` y `DECISIONES.md`.

No modifiquen `main.py`, `server.js`, `worker.py`, `index.html`, `default.conf`, `db/init.sql` ni `tools/verify.py`. PostgreSQL y Redis usan imágenes oficiales: no necesitan Dockerfile.

6. Plan de trabajo

Minutos	Actividad	Resultado comprobable
0–10	Formación del equipo, asignación del ZIP y diagnóstico	Equipo identificado; herramientas verificadas; primera revisión estática guardada
10–35	Dockerfiles heredados	`catalog` y `orders` construyen
35–65	Dockerfiles nuevos	`frontend` y `worker` construyen
65–75	Revisión cruzada	Usuario, salud, secretos, tamaño y caché revisados
75–105	Integración con Compose	Seis servicios sanos; redes, volumen y dependencias configurados
105–115	Prueba funcional	Productos visibles y pedido procesado
115–120	Cierre	Verificador, pendientes y commit del punto de control

7. Paso a paso

Paso 1. Comprobar el entorno

```
docker --version
docker compose version
docker run --rm hello-world
python3 --version
python3 tools/verify.py --static
```

Si falla una herramienta, avisen al profesor antes de editar código.

Paso 2. Corregir `catalog` y `orders`

En cada Dockerfile:

1. Copien primero el archivo de dependencias y luego instalen dependencias; copien el código al final para aprovechar la caché.
2. Usen al menos dos etapas en `catalog` y `orders`.
3. Dejen compiladores y cabeceras fuera de la imagen final.
4. Ejecuten el proceso como usuario no root.
5. No escriban contraseñas ni `TEAM_SECRET` en `ARG`, `ENV` o `RUN`.
6. Definan un `HEALTHCHECK` que pruebe el servicio real.

Prueben cada servicio antes de continuar:

```
docker compose build catalog
docker compose build orders
```

Paso 3. Crear `frontend/Dockerfile` y `worker/Dockerfile`

Para `frontend`, utilicen una imagen Nginx no privilegiada que escuche en 8080. Para `worker`, instalen sus dependencias y comprueben la antigüedad de `/tmp/healthy`; el proceso no expone HTTP.

Creen un `.dockerignore` no vacío en los cuatro servicios. Como mínimo excluyan Git, cachés, dependencias locales y archivos temporales que no necesita la imagen.

Paso 4. Completar `docker-compose.yml`

Requisito	**Comprobación**
Puerto	Solo `frontend` publica `${FRONTEND_PORT}:8080`
Configuración	Las aplicaciones y la base leen `.env`; los secretos no forman parte de las imágenes
Persistencia	Un volumen nombrado se monta en `/var/lib/postgresql/data`
Inicialización	`db/init.sql` se monta como solo lectura en `/docker-entrypoint-initdb.d/`
Redes	`frontend` y `db` no comparten ninguna red
Salud	`db` y `cache` tienen *healthcheck*; las cuatro aplicaciones también
Orden	`catalog`, `orders` y `worker` esperan dependencias con `condition: service_healthy`

Paso 5. Levantar y diagnosticar

```
docker compose config
docker compose up -d --build --wait
docker compose ps
docker compose logs --tail=80 catalog orders worker
```

No usen `sleep` como solución. Si algo queda `unhealthy`, revisen su healthcheck y sus logs. Recuerden que `localhost` dentro de un contenedor apunta al propio contenedor.

Paso 6. Verificar y cerrar

```
python3 tools/verify.py --no-cache-check
python3 tools/verify.py
git status
git add -A
git commit -m "checkpoint dia 1"
```

Antes de irse, anoten en `DECISIONES.md`: decisiones ya tomadas, dos problemas diagnosticados y pendientes concretos para la segunda sesión. Pueden apagar sin perder el volumen mediante `docker compose down`. No usen `down -v`.

8. Lista de salida

- ☐ Los cuatro Dockerfiles existen y construyen.
- ☐ Los cuatro `.dockerignore` existen.
- ☐ Los seis servicios aparecen en `docker compose ps`.
- ☐ El frontend y la base no comparten red.

☐ Un pedido llega a `processed`.

☐ Hay un commit `checkpoint dia 1` y no quedan cambios ajenos sin integrar.